MACHINE LEARNING

Machine Learning :

ML is method where computers learn patterns from data instead of being explicity programmed. (Input + Output = Learn Rules). Machine learns the rule by itself.

For example:

Spam detecting : instead of writing 1000 rules like

- If message contains “lottery”
- If message contains “win money”

We give:

- Thousands of emails
- Label them Spam / Not Spam Model automatically learns pattern.

In real world writing rules is hard, data is huge, patterns are complex. So, ML learns hidden relationships automatically.

Types of Machine Learning

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

Supervised Learning:

Model learns from labelled data. Labelled means we know the correct answer.

For example:

Predict House Price : Price is known. Model learns relationship between (Size, Bedrooms) – price.

Types :

1. Regression : Output is continuous number.

   Ex: House price prediction, Salary Prediction, Temperature prediction

   Algorithms : Linear Regression, Random Forest Regressor.

2. Classification: Output is category.
   Ex: Spam/ Not Spam, Disease / No Disease, Purchased / Not Purchased.
   Algorithms: Logistic Regression, SVM, KNN, Decision Tree.

Unsupervised Learning:

Model learns from data without labels.

For example:

Customer segmentation.

You have:

| Age | Income | Spending |

But no label like "Rich" or "Poor".

Model automatically groups similar customers.

Types:

1. Clustering: Group similar data points.
   Algorithms: K-Means, Hierarchical Clustering
2. Dimensionality Reduction: Reduce number of features.
   Algorithm: PCA

Train/Test Split:

We divide data into two parts: Training Data (Model learns from this), Testing Data(We check how well model performs on unseen data)

Ex: suppose we have 1000 rows of data. We split 80%(Training-800rows), 20%(testing-200rows)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Overfitting & Underfitting :

1. Overfitting: Model performs very well on training data and poorly on test data.
   - Ex: Training Accuracy (98%), Test Accuracy(65%).
   - It happens when model too complex, too many features, weak regularization, too deep decision tree. Here model memorized data instead of learning pattern.
   - To avoid overfitting by increase regularization, get more data, reduce model complexity, use cross-validation.
2. Underfitting: Model performs poorly on both Training Accuracy(60%), Test Accuracy(58%).
   - It happens model too simple, Strong regularization, Not enough trainig, important features missing.
   - To avoid underfitting by decrease regularization, increase model complexity, add useful features.

Regularization:

we use regularization when the model is overfitting. It tells the model don't make weights too large. So, regularization reduces complexity. We should use this when then model performance gap between train and test is large, Dataset is small, Features are many and you suspect overfitting.

Regularization mostly used in Linear Regression, Logistic Regression, Neural Networks, SVM. If training accuracy is greater than test accuracy we have to increase the regularization. If training accuracy is lower than test accuracy we have to decrease the regularization. Mainly it is used to control overfitting and balance model complexity.

It is not necessary when we use Decision Trees and Random Forest. Because the model complexity comes from large weights. So, we control complexity by penalty on weights. But in decision trees does not use weights instead it works on the rules. Sometimes if tree grows too much until each leaf has 1 sample, perfectly memorize training data and overfit badly. Instead of using weight penalty, we control max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes (these are called structural regularization).

Bias and Variance :

1. High bias comes (Underfitting) : it comes when your model is too simple to capture real patterns. Model is too simple and can't even learn training data properly. Make model strong by increasing complexity, reduce regularization, increase max_depth.
2. High Variance(Overfitting): Model learned training data too perfectly. But can't handle new data. Make model simpler by increase regularization, reduce max_depth, reduce complexity.
3. Balance Bias & Variance : doesn't memorize just understands the logic.

Evaluation Metrics:

After building a model, we must answer how good is my model? We measure this using evaluation metics.

1. Confusion Matrix :

   Actual Positive + Predicted Positive = True Positive (AP + PP = TP)

   Model predicted spam. It is actually spam.

   Actual Positive + Predicted Negative = False Negative (AP + PN = FN)

   Model predicted not spam. But actually spam.

   Actual Negative + Predicted Positive = False Positive (AN + PP = FP)

   Model predicted spam. But actually not spam.

   Actual Negative + Predicted Negative = True Negative (AN + PN = TN)

   Model predicted not spam. It is actually not spam.
2. Accuracy:

Use when data is balanced (50% yes, 50% no), Both types of mistakes are equally okay.
Ex: Predict male/female, Predict pass/fail (balanced dataset)

3. Precision:
   Use when false positive is dangerous.
   Ex: Spam detection(Don't' block important email), Face recognition in phone(Don't unlock for wrong person), Police system (Don't accuse innocent person). So, when false alarm is costly – use precision.
4. Recall:
   Use false negative is dangerous. If model misses a real positive case, it is risky.
   Ex: Cancer detection(Don't' miss patient). Fraud detection(Don't miss fraud transaction), Fall detection.
5. F1-score:
   Use when dataset is imbalanced, both precision & recall matter.
   Ex: Fraud detection, medical diagnosis.

Supervised Algorithms :

Linear Regression: y = mx+b [x(input, feature), y(output, label, target), m(slope), b(intercept)]

Use when the output is a number (continuous value), Relationship is approximately linear. Continuous output + linear relationship = Linear Regression

Ex: predict house price, salary, temperature.

MSE = Mean Squared Error. Its purpose is to measure how wrong our regression model is.

How far predicted values are from actual values. That number is MSE.

MSE is used as a loss function in regression to measure the average squared difference between predicted and actual values, and the model is trained to minimize this error.

Big mistakes are very dangerous.

Example:

Predicting:
Airplane fuel requirement 🛫

If model makes huge mistake:
Plane may crash.

So we want to punish big errors strongly.

Use MSE or RMSE.

You don't want big mistakes to dominate.

Example:

Predicting:
Daily sales of a shop.

If one day prediction is very wrong,
it's okay.

So use MAE.

House prices increase when size increases.(0-Bad, 0.5-Medium, 0.7-Good, 0.9-very good, 1-perfect)

If your model captures that relationship well:

R^2 will be high.

If your model cannot understand relationship:

R^2 will be low.

Classification: output is categorical [pass/fail, spam/not spam]

Logistic Regression : It predicts the probability of belonging to a class and then converts it into 0 or 1. Bias = extra constant added so model is not forced to pass through zero. Sigmoid converts any number into value between: 0 and 1.

Usually threshold = 0.5

If probability ≥ 0.5 → Class = 1
If probability < 0.5 → Class = 0

Input Features
→ Linear Score (wx + b)
→ Sigmoid
→ Probability
→ Apply 0.5 rule(threshold)
→ Final Class (0 or 1)

It looks like an S shape:

- Left side → near 0
- Middle → 0.5
- Right side → near 1

That smooth S-shape is why it's called logistic curve.

KNN: KNN classifies a new data point based on the majority class of its K nearest neighbors.

If K = 1: Very sensitive to noise (overfitting), If K is very large: Too general (underfitting)

Usually choose odd number like: 3,5,7.

If one feature is large (like salary in lakhs), and another is small (like age), Large feature will dominate distance. So we use: StandardScaler.

Euclidean Distance means: The straight-line distance between two points.

Student A study hours = 2
Student B study hours = 5

Distance = |5 - 2| = 3

In one dimension, it is just difference.

StudyHours and Attendance

Point A = (2, 60)
Point B = (5, 80)

Distance: $\sqrt{(5-2)^2 + (80-60)^2}$
= √(9 + 400)
= √409

That is Euclidean distance. KNN decides class based on nearest neighbors.

Nearest = smallest distance.

So distance calculation is very important.

SVM: SVM is a supervised learning algorithm used for:

Classification
Regression

SVM finds the best decision boundary (hyperplane) that separates two classes with maximum margin. The line (or plane) that separates two classes., 2D → line, 3D → plane, Higher dimension → hyperplane

Distance between the decision boundary and the nearest data points.

- SVM tries to maximize this margin. Small margin → model becomes sensitive
- Large margin → model becomes stable
- Large margin → better generalization
- Reduces overfitting

SVM always chooses the safest boundary.

Linear SVM

Used when data can be separated by a straight line.

2⃣Non-Linear SVM

Used when data cannot be separated by a straight line.

Uses Kernel Trick: **Linear Kernel** Used when data is already linearly separable. It draws a straight line.

Polynomial Kernel Used when boundary is curved (but not too complex). It creates polynomial curves.

RBF Kernel (Radial Basis Function) Most powerful and most used. It can create very flexible boundaries.

Imagine separating fruits:

🍎 Apples inside circle
🍌 Bananas outside circle

Linear → cannot separate
Polynomial → maybe
RBF → easily separates

Select the parameter : C**,** Gamma (if RBF)

Small C (Soft Teacher)

- Allows small mistakes
- Doesn't try to perfectly classify every point
- Keeps boundary wide
- Model becomes simple
- Less overfitting

Large C (Strict Teacher)

- Does not allow mistakes
- Tries to classify all training points correctly
- Boundary becomes tight
- Model becomes complex
- Can overfit

  C = Strictness level
  Small C → Relaxed
  Large C → Strict

Small Gamma

- Light spreads far
- Points influence large area
- Boundary becomes smooth
- Model simple

✕ Large Gamma

- Light spreads only near the point
- Each point creates small sharp influence
- Boundary becomes very wiggly
- Model complex
- Can overfit

  Gamma = Curve complexity
  Small Gamma → Smooth boundary
  Large Gamma → Very complex boundary

Small C + Small Gamma	Very simple model
Large C + Large Gamma	Very complex model
Large C + Small Gamma	Strict but smooth
Small C + Large Gamma	Relaxed but complex

Overfitting means model is too complex.

Then:

👉 Reduce C
👉 Reduce Gamma

Underfitting means model is too simple.

Then:

👉 Increase C
👉 Increase Gamma

Gamma controls the **zoom level**.

Small Gamma → Zoomed out (simple view)
Large Gamma → Zoomed in (very detailed, complex)

Naïve Bayes : it is a classification algorithm based on probability. It uses bayes theorem and probability rules. It predicts the class with highest probability.

Why it is called Naïve? Because it assumes all features are independent of each other. Even if they are not fully independent in real life.

First it checks the P(Spam|words) then compares with P(Not Spam|Words). Which ever is higher that class is predicted. Bayes theorem formula $P(A|B)=(P(B|A)*P(A))/P(B)$.

It calculates the prior probability, calculate likelihood (probability of words given spam), Apply bayes formula, compare probabilities and choose class with highest probability.

Types of Naïve Bayes :

1. Gaussian Naïve Bayes – for continuous data

2. Multinomial Naïve Bayes – for text data (word counts)
3. Bernoulli Naïve Bayes – for binary features(0/1)

Decision Trees:

It is a supervised machine learning algorithm used for classification and regression. It makes decisions using a tree like structure. We called it as a tree because it has a root node(top), branches(conditions), Leaf nodes(final output).

Should I pay outside? Tree may look like: is it raining? Yes-Don't play, No-check temperature. Is temperature high? Yes-play, No-Don't play. It keeps asking questions step by step. That's a Decision tree.

Working of Decision Tree :

Take full dataset, the model aks which feature best separates the data?, Data is divided into branches and Each branch contains smaller subset. For each branch again choose best feature and split again repeat this process until stopping condition reached. Leaf node gives final prediction.

Criteria for Best Split : for classification Gini Index, Entropy these measure impurity. Less impurity = better split.

When splitting data, the tree asks:

"Which feature gives the purest split?"

To measure purity, we use:

- Gini Index
- Entropy

Gini measures how often a randomly chosen element would be incorrectly classified.

Simple understanding:

- If all values belong to one class → Gini = 0
- If classes are evenly mixed → Gini is high

Range:

0 → Pure
0.5 (binary case) → Maximum impurity

Entropy measures disorder or randomness.

It comes from information theory.

Formula (just for reference)

$$Entropy = -p1log2(p1) - p2log2(p2)$$

Don't fear the formula.

Understand meaning:

- If all data in one class → Entropy = 0
- If classes are evenly mixed → Entropy = 1 (maximum in
- Gini:
- ✓ Simpler formula
  ✓ Faster to compute
- Entropy:
- ✓ Uses logarithm
  ✓ Slightly slower
- That's why:
- 👉 In practice, sklearn uses Gini as default.

Gini measures how mixed the classes are. If node has: All Pass – pure, Half pass, Half fail – impure.

Random Forest : it is an ensemble learning algorithm. It combines Many decision trees to make one strong model. It randomly selects data and features that's why it is called Random Forest.

Creates many trees and each tree gives prediction and the final output = manority voting (for classification).

Many weak models combined = One strong model. This reduces variance and overfitting.

Bootstapping and Feature Randomness make random forest powerful. Bootstrapping → Different data, Bootstrapping → Different data
Feature randomness → Different way of thinking so trees become independent and independent trees- better final prediction. Bootstrapping = Random data for each tree
Feature randomness = Random features for each split

Both make trees different.

ML Mathematics

◻Cost Function

What is it?

Cost function = A number that tells
how wrong your model predictions are.

Example:

You predict house price = 5 lakh
Actual price = 8 lakh

Model is wrong.

Cost function calculates that error.

When do we use it?

👉 Every time we train a model.

Because without measuring error,
model doesn't know how to improve.

In code:

model.fit(X, y)

Internally:

- Model predicts
- Calculates cost
- Tries to reduce it

You don't call it manually.
It always exists.

2️⃣ Gradient Descent

What is it?

It is the method used to reduce the error.

Model:

1. Predicts
2. Checks error
3. Changes weights
4. Repeats

That "changing weights again and again" is gradient descent.

When do we use it?

👉 When model has many features
👉 When we cannot calculate best solution directly

Used in:

- Linear Regression
- Logistic Regression
- Neural Networks
- Deep Learning

Almost everywhere.

3️⃣ Convex Function

What is it?

Convex means:

The error graph has only ONE minimum point.

So model can safely find best solution.

When does it matter?

It matters in:

- Linear Regression
- Logistic Regression

These have convex loss → easy optimization.

It becomes important in:

- Neural Networks

Because they are non-convex (many minimums).

4⃣ Regularization

Now very important.

What is it?

Regularization prevents overfitting.

Overfitting = Model performs:

- 99% on training
- 60% on testing

Model memorized data.

For regression:

- Start with Linear Regression
- Then try Random Forest Regressor

For classification:

- Start with Logistic Regression
- Then try Random Forest
- Then try SVM

Compare accuracy.

Unsupervised Learning : in this we only have input data, no correct answers, no labels. We don't know categories we just want to find patterns.

Ex: Suppose you have customer data: Age, Income, Spending score. But you don't know which customer belongs to which group. Now we have to group similar customers together that is called clustering.

1⃣What is K-Means?

K-Means is an **unsupervised learning algorithm** used to:

👉 Group similar data points into clusters.

It divides data into **K number of clusters**.

K = number of groups.

◈ 2⃣Why Do We Use K-Means?

We use K-Means when:

- No target/output variable
- We want to group similar data
- We want to find hidden patterns

◈ 3⃣How K-Means Works (Flow)

Step 1: Choose K

Select number of clusters (example: K = 3)

Step 2: Select Centroids

Randomly pick K points as cluster centers.

Step 3: Calculate Distance

Calculate distance between each data point and centroid
(Usually Euclidean distance)

Step 4: Assign Cluster

Assign each point to the nearest centroid.

Step 5: Update Centroid

Recalculate new centroid (mean of cluster points).

Step 6: Repeat

Repeat steps 3–5 until centroids stop changing.

◈ 4⃣ Distance Formula (Euclidean Distance)

$$d = sqrt((x2 - x1)^2 + (y2 - y1)^2)$$

-10-8-6-4-2246810-10-5510(6.0, 6.0)(-6.0, -6.0)d = 16.97

Used to measure similarity between points.

◈ 5⃣ Objective of K-Means

Minimize **Within Cluster Sum of Squares (WCSS)**

It tries to make:
✓ Points inside cluster very close
✓ Clusters very compact

◈ 6⃣ Advantages

✓ Simple and fast
✓ Works well for large datasets
✓ Easy to understand

◈ 7⃣ Disadvantages

✕ Need to choose K manually
✕ Sensitive to outliers
✕ Works well only for spherical clusters
✕ Different initialization → different results

◈ 8⃣ How to Choose K?

Elbow Method

The Elbow Method is used to determine the optimal number of clusters (K) in K-Means clustering.

Since K-Means requires us to choose K manually, the Elbow Method helps in selecting the best value.

◈ 2⃣ Purpose of Elbow Method

👉 To find the best number of clusters
👉 To avoid choosing too many or too few clusters

◈ 3. Concept Behind Elbow Method

K-Means minimizes:

WCSS (Within Cluster Sum of Squares)

WCSS = Sum of squared distances between each data point and its cluster centroid.

Small WCSS = Compact clusters.

◈ 4. How It Works (Steps)

1. Run K-Means for different values of K (e.g., 1 to 10).
2. For each K, calculate WCSS.
3. Plot graph:

- X-axis → Number of clusters (K)
- Y-axis → WCSS

4. Identify the point where the curve sharply bends.
That point is called the Elbow Point.

◈ 5. Why It Is Called "Elbow"?

Because the graph looks like a human arm bending at the elbow.

WCSS decreases rapidly at first,
then decreases slowly after some point.

That bending point = Best K.

◈ 6. Graph Interpretation

When K increases:

- WCSS always decreases.
- Large decrease initially.
- After optimal K, improvement becomes very small.

Choose K at the bending point.

◈ 7⃣ Example

K	WCSS
1	1000
2	600
3	300
4	250
5	230

Best K = 3
Because after 3, reduction in WCSS is small.

◈ 8⃣ Advantages

✓ **Simple and easy to understand**
✓ **Helps avoid over-clustering**
✓ **Widely used in practice**

◈ 9⃣ Limitations

✕ **Sometimes graph has no clear elbow**
✕ **Subjective (depends on visual interpretation)**
✕ **Not always reliable**

◈ 🔟 When To Use

✓ **Before applying K-Means**
✓ **When number of clusters is unknown**
✓ **In customer segmentation problems**

◈ 1⃣2⃣ Real-Life Applications

✓ Customer Segmentation
✓ Market Analysis
✓ Image Compression
✓ Fraud Detection
✓ Document Clustering

◈ 1⃣0⃣ Data Preprocessing Required?

Yes ✅

Before K-Means:

✓ Handle missing values
✓ Remove outliers
✓ Feature scaling (Very Important)

Because K-Means uses distance.

Hierarchical clustering: Hierarchical Clustering is an **unsupervised learning algorithm** that groups similar data points into clusters by building a hierarchy (tree structure) of clusters.

The result is represented using a **Dendrogram**.

◈ 2⃣ Key Idea

Instead of directly dividing data into K clusters (like K-Means),

Hierarchical clustering:

- Either merges smaller clusters step-by-step
- Or splits bigger clusters step-by-step

◈ 3⃣ Types of Hierarchical Clustering

1⃣ Agglomerative (Bottom → Up) ✅ (Most Common)

- Start with each data point as a separate cluster
- Merge closest clusters
- Repeat until all points become one cluster

Small → Big

2⃣ Divisive (Top → Down)

- Start with all data in one cluster
- Split into smaller clusters
- Repeat

Big → Small

◈ 4⃣ Distance Methods

What is a Distance Method?

A distance method is used to measure **how far two data points are from each other**.

In clustering:

- Smaller distance → More similar
- Larger distance → Less similar

Used in:

- K-Means
- Hierarchical Clustering
- KNN
- Recommendation systems

◈ 2️⃣ Euclidean Distance (Most Common)

📏 Straight-line distance between two points.

$$d = sqrt((x2 - x1)^2 + (y2 - y1)^2)$$

-10-8-6-4-2246810-10-5510(6.0, 6.0)(-6.0, -6.0)d = 16.97

Real-Life Example

Imagine two houses on a map.

You measure the direct straight-line distance between them.

Used in:

- K-Means clustering
- Customer segmentation
- Numerical data

✓ Works best when data is continuous
✓ Most commonly used distance

◈ 3️⃣ Manhattan Distance

🚶 Distance measured along horizontal and vertical paths.

$$d = | x2 - x1 | + | y2 - y1 |$$

Real-Life Example

In a city like New York:

You cannot walk diagonally through buildings.

You move:

- 3 blocks right
- 2 blocks up

Total distance = 5 blocks

Used in:

- Grid-based systems
- When data has outliers
- High-dimensional data

✓ More robust to extreme values than Euclidean

◈ 4⃣ Minkowski Distance

General form of distance.

$$d = (|x2 - x1|^p + |y2 - y1|^p)^{(1/p)}$$

If:

- p = 1 → Manhattan
- p = 2 → Euclidean

Real-Life Example

Think of Minkowski as a "family" of distances.

You choose p depending on problem requirement.

Used when:

- Flexibility is needed
- Different sensitivity required

◈ 5⃣ Cosine Distance

📐 Measures similarity based on angle between vectors.

Formula based on dot product:

$$cos(\theta) = (A \cdot B)/(||A||||B||)$$

Real-Life Example

Two students:

Student A: [90, 90]
Student B: [45, 45]

Marks different in magnitude
But pattern same

Cosine says:
👉 They are similar (same direction)

Used in:

- Text clustering
- Document similarity
- Recommendation systems
- NLP

✓ Best for high-dimensional sparse data

◈ 6⃣ Hamming Distance

Measures number of different positions.

Real-Life Example

Password comparison:

101110
100100

Different positions = 2

Used in:

- Categorical data
- Binary data
- DNA sequence analysis

◈ 5⃣ Linkage Methods

What is a Linkage Method?

A linkage method is used in **Hierarchical Clustering** to measure the distance between two clusters.

👉 Distance method = distance between two points
👉 Linkage method = distance between two clusters

When merging clusters, linkage decides:

“Which clusters should be combined?”

◈ 2️⃣ Types of Linkage Methods

1️⃣ Single Linkage (Nearest Neighbor)

📌 Definition:
Distance between two clusters is the **minimum distance** between any two points from the clusters.

Simple Meaning:

Take the closest pair of points from two clusters.

Real-Life Example:

Two friend groups.

If even one person from Group A is very close to one person in Group B,

The two groups merge.

Advantage:

✓ Can detect irregular shaped clusters

Disadvantage:

✕ Forms chain-like clusters (chaining effect)

2️⃣ Complete Linkage (Farthest Neighbor)

📌 Definition:
Distance between two clusters is the **maximum distance** between any two points.

Simple Meaning:

Take the farthest pair of points from two clusters.

Real-Life Example:

Two teams merge only if **all members are close** to each other.

If even one member is far away → they do not merge.

Advantage:

✓ Creates compact clusters

Disadvantage:

✕ Sensitive to outliers

5⃣ Average Linkage

📌 Definition:
Distance between two clusters is the **average distance** between all pairs of points.

Simple Meaning:

Calculate average closeness between clusters.

Real-Life Example:

Two departments merge if the average interaction between employees is strong.

Advantage:

✓ Balanced method
✓ Less extreme than single or complete

4⃣ Ward's Method (Most Common)

📌 Definition:
Merges clusters in a way that **minimizes total within-cluster variance**.

It tries to keep clusters as compact as possible.

Simple Meaning:

Merge clusters only if the new cluster is very tight and similar inside.

Real-Life Example:

Forming classrooms:

Students are grouped so that students inside one class are very similar in marks.

Advantage:

✓ Produces well-separated clusters
✓ Most commonly used
✓ Works very well with Euclidean distance

🔷 6⃣ Dendrogram

A tree diagram that shows:

- How clusters merge
- At what distance they merge

You choose number of clusters by cutting the dendrogram at a certain height.

◈ 7⃣ Algorithm Steps (Agglomerative)

1. Calculate distance between all data points
2. Merge closest two clusters
3. Recalculate distances
4. Repeat until one cluster remains

◈ 8⃣ Advantages

✓ No need to specify K initially
✓ Provides hierarchical structure
✓ Easy to visualize (dendrogram)

◈ 9⃣ Disadvantages

✕ Slow for large datasets
✕ Cannot undo a merge once done
✕ Sensitive to noise and outliers

◈ 🔟 When To Use

✓ Small to medium datasets
✓ When number of clusters is unknown
✓ Research analysis
✓ Need hierarchical structure

DBSCAN : **Definition**

DBSCAN is an **unsupervised clustering algorithm** that groups data points based on **density** and identifies outliers (noise).

It forms clusters in regions where data points are densely packed and separates regions with low density.

◈ 2⃣ Key Idea

- Dense region → Cluster

- Sparse region → Noise

Unlike K-Means:

- ✕ No need to specify number of clusters (K)
- ✓ Automatically detects outliers

◈ 5️⃣ Parameters of DBSCAN

DBSCAN uses two main parameters:

1️⃣ Epsilon (ε)

What is it?

Epsilon is the radius (distance limit) around a point.

It answers:

👉 "How close should other people be to consider them neighbors?"

Real-Life Example

Imagine you draw a circle of 2 meters radius around a person.

If other people are inside that circle → they are neighbors.

If outside → not neighbors.

What happens if:

◈ ε is too small
→ Very few neighbors
→ Many people become noise

◈ ε is too large
→ Everyone becomes one big cluster

2️⃣ MinPts (Minimum Points)

What is it?

Minimum number of neighbors required inside ε radius to form a cluster.

It answers:

👉 **"How many people are needed to call it a group?"**

Real-Life Example

Suppose:

MinPts = 4

That means:

Inside the 2-meter circle,
there must be at least 4 people.

Then only it becomes a group.

If:

MinPts is small (like 2)
→ Many small groups form

MinPts is large (like 10)
→ Only big dense groups form

How to Choose MinPts?

✓ Rule 1 (Standard Rule)

MinPts ≥ (Number of Dimensions + 1)

Example:

- **2D data → MinPts ≥ 3**
- **3D data → MinPts ≥ 4**
- **5 features → MinPts ≥ 6**

✓ Rule 2 (Practical Use)

In most real-world problems:

MinPts = 4 or 5

This works well for many datasets.

✅ **Why Not Too Small or Too Large?**

- **Very small MinPts → Many small clusters form**
- **Very large MinPts → Many points become noise**

MinPts controls how strict clustering should be.

◈ 2⃣ Choosing Epsilon (ε) Using K-Distance Graph

Epsilon is chosen using a graphical method called:

✅ **K-Distance Graph Method**

◈ What is K-Distance?

For each data point:

1. **Find its K nearest neighbors**
2. **Take distance to the K-th nearest neighbor**

Where:

K = MinPts

◈ Steps to Choose ε

✅ Step 1: Choose MinPts

Suppose:

MinPts = 4

Then compute distance to 4th nearest neighbor.

✅ Step 2: Compute K-distance for All Points

For every point, calculate:

Distance to its 4th nearest neighbor.

✅ Step 3: Sort Distances (Ascending Order)

Sort all K-distances from smallest to largest.

✅ Step 4: Plot the Graph

- **X-axis → Sorted points**
- **Y-axis → K-distance**

The graph will show:

- **Gradual increase (cluster points)**
- **Sudden sharp increase (noise points)**

✅ Step 5: Find the Elbow Point

The point where the graph sharply increases is called:

👉 Elbow point

The Y-value at elbow = Best ε value.

◈ Why Does Elbow Appear?

- **Points inside clusters have small neighbor distances**
- **Noise points have large neighbor distances**
- **The sudden jump separates clusters from noise**

◈ What Happens If ε Is Wrong?

- **ε too small → Many points become noise**
- **ε too large → All clusters merge into one**

So elbow gives natural separation.

◈ 4⃣ Types of Points

DBSCAN classifies points into three types:

1⃣ Core Point

A person is a core point if:

Inside ε radius,
there are ≥ MinPts people.

Real-Life Example

Imagine one person standing in middle of a crowd.

Within 2 meters,
there are 6 people.

MinPts = 4

Since 6 ≥ 4
👉 That person is a Core Point.

Core points are center of groups.

2️⃣ Border Point

A person is a border point if:

- **They are close to a core point**
- **But they do NOT have enough neighbors themselves**

Real-Life Example

Imagine someone standing at edge of group.

Inside their circle,
only 2 people.

MinPts = 4
So they are not core.

But they are close to a core person.

👉 So they become Border Point.

They belong to cluster,
but are not strong enough to form cluster alone.

3️⃣ Noise Point (Outlier)

A person is noise if:

- **They are not core**

- **Not close to any core point**

Real-Life Example

One person sitting alone on bench.

Inside 2 meters,
no one else.

Not near any group.

👉 **That person is Noise.**

In sklearn:
Noise points are labeled as -1

◈ 5️⃣ Working Steps of DBSCAN

1. Select a point
2. Find neighbors within ε
3. If neighbors ≥ MinPts → Create cluster
4. Expand cluster by checking neighbors of neighbors
5. Repeat
6. Unassigned points → Noise

◈ 6️⃣ Advantages

✓ No need to specify number of clusters
✓ Detects outliers automatically
✓ Works well for irregular-shaped clusters
✓ Good for spatial data

◈ 7️⃣ Disadvantages

✕ Difficult to choose optimal ε
✕ Not suitable for varying density datasets
✕ Struggles with high-dimensional data

◈ 8️⃣ When to Use DBSCAN

✓ When dataset contains noise
✓ When clusters are irregular shaped
✓ Geographic data analysis

✓ Fraud detection
✓ Anomaly detection

PCA (principal component analysis):

1⃣ Definition

PCA (Principal Component Analysis) is an unsupervised dimensionality reduction technique that transforms correlated features into a smaller number of uncorrelated variables called principal components, while preserving maximum variance.

◈ 2⃣ Purpose of PCA

- **Reduce number of features**
- **Remove multicollinearity**
- **Reduce computation time**
- **Improve model performance**
- **Help in visualization**
- **Reduce noise**

◈ 3⃣ Key Idea

PCA finds new directions (axes) in data where:

- **Data variance is maximum**
- **Information is preserved**

These new directions are called:

👉 Principal Components (PCs)

◈ 4⃣ Principal Components

- **Linear combination of original features**
- **Uncorrelated with each other**
- **Ordered by importance**

Order of Components:

- **PC1 → Maximum variance**
- **PC2 → Second highest variance**
- **PC3 → Third highest variance**

◈ 5⃣ Important Concepts

✓ Variance

Measure of data spread.
More variance = more information.

✓ Eigenvectors

Directions of principal components.

✓ Eigenvalues

Amount of variance captured by each component.

Higher eigenvalue → More important component.

◈ 6️⃣ Steps in PCA

1️⃣ Standardize the data
2️⃣ Compute covariance matrix
3️⃣ Calculate eigenvalues & eigenvectors
4️⃣ Sort components by eigenvalues
5️⃣ Select top components
6️⃣ Transform original data

◈ 7️⃣ Explained Variance Ratio

Shows how much information each principal component retains.

Example:

PC	Variance Explained
PC1	60%
PC2	25%
PC3	10%

Cumulative variance helps decide number of components.

Usually keep components that explain:

👉 80% – 95% variance

◈ 8️⃣ Advantages of PCA

✓ Reduces dimensionality
✓ Removes correlated features
✓ Improves model efficiency

✓ **Reduces overfitting**
✓ **Useful for visualization**

◈ 9️⃣ Disadvantages of PCA

✕ **Loss of interpretability**
✕ **Possible information loss**
✕ **Not suitable for categorical data**
✕ **Sensitive to scaling**

◈ 🔟 When to Use PCA

✓ **Large number of features**
✓ **Highly correlated features**
✓ **Before clustering**
✓ **Before regression**
✓ **For 2D/3D visualization**